

Hands-on Introduction to Computer Architecture with RISC-V

ASSIGNMENT 04

2 Stage Pipelined RISC-V Processor

NAME : SHANIKA KAVINDI

REG.NO : SKF 2500222

DATE : 2025/12/27

1. Objective

Implement and verify a 2-stage pipelined version of the provided single-cycle RV32 processor. Stage-1 performs IF/ID/EXE and Stage-2 performs MEM/WB. A Fibonacci(n=12) program is executed and verified in simulation; then execution time and maximum clock frequency are compared against the single-cycle implementation. An optional forwarding mode is also implemented.

2. Pipeline structure

The pipelined CPU is implemented in src/processor.sv. A reference single-cycle CPU is kept in src/processor_singlecycle.sv. The pipeline boundary is placed after the EXE stage output so that Stage-2 contains the data-memory access and the register writeback.

Signals stored in the Stage-1 -> Stage-2 pipeline register

Signal	Width	Purpose
rd_s2	5	Destination register number
funct3_s2	3	Load/store size & sign control in Stage-2
regWriteEn_s2	1	Enables Stage-2 register writeback
isMemRead_s2	1	Selects MEM read and WB data = memRead
isMemWrite_s2	1	Enables MEM write in Stage-2
aluOut_s2	32	Effective address (loads/stores) or ALU result (R/I/branch/jump target)
wb_pre_mem_s2	32	Writeback data before MEM mux (ALU result or PC+4 for JAL/JALR)
memWrite_s2	32	Formatted store data (bytes positioned for SB/SH/SW)

3. Pipeline hazards discussion

Data hazards (RAW)

Stage-1 reads the register file while Stage-2 writes results. A back-to-back dependency (producer in Stage-2, consumer in Stage-1) can cause Stage-1 to read an old value unless software inserts independent instructions / NOPs, or hardware forwarding is used.

Control hazards

minimized. Branch/JAL/JALR targets are computed in Stage-1, so the PC can be redirected immediately at the next clock edge. Because this simple 2-stage design does not speculatively fetch beyond Stage-1, no branch delay slot is required.

Structural hazards

possible in principle because instruction fetch and data memory share the same mainMemory array (unified memory). In a real implementation this would require either separate instruction/data memories or arbitration. In this simplified RTL model the array is read combinationally for fetch and read/write in Stage-2 without stalling.

4. Fibonacci program + hazard avoidance

The program computes Fibonacci(n) with $n = 12$, first storing n to memory location 0x80 and loading it back. The final result $F(12)=144$ is written to x3 (gp) and x10 (a0).

4.1 Program files

- programs/program_singlecycle.txt/.hex: No hazard avoidance (used for single-cycle and forwarding demo).
- programs/program_pipelined.txt/.hex: Software hazard avoidance for the pipelined CPU with no forwarding (one filler instruction after the load and small scheduling changes).

4.2 Key hazard-avoidance changes (software scheduling)

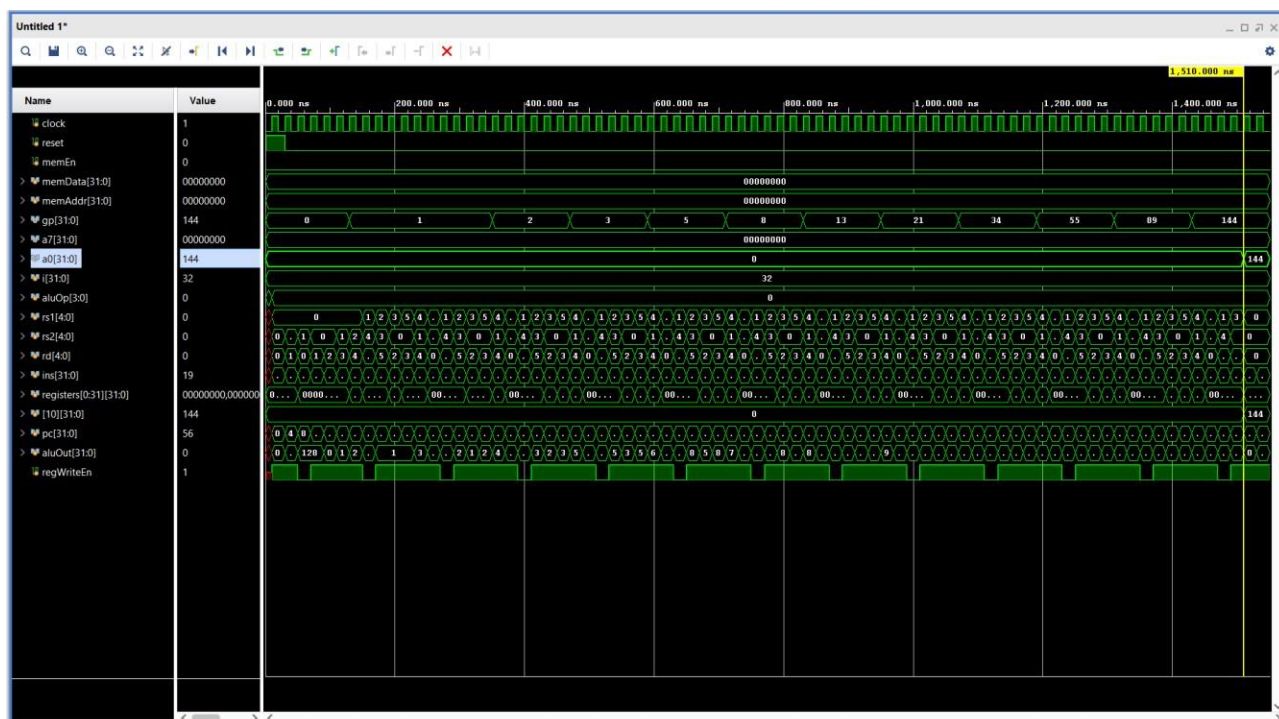
- Inserted an independent instruction after the load of x1 so that the loaded n value is written back before the following blt reads it.
- Reordered updates inside the loop so that the temporary value (x5) is not consumed immediately in the next cycle in the 2-stage pipeline.

5. Simulation

Three testbenches are provided in tb/ to match the required comparisons. Each testbench preloads the program hex into DUT memory using \$readmemh, resets the DUT, then runs until the PC reaches the final jal x0, 0 instruction and a0 equals 144.

Simulation waveforms

➤ *Single-cycle CPU*



project_1 - [C:/Users/lapto/OneDrive/Desktop/A4_submission/project_1/project_1.xpf] - Vivado 2025.1

File Edit Flow Tools Reports Window Layout View Run Help Quick Access Running synth_design Cancel

Flow Navigator SIMULATION - Behavioral Simulation - Functional - sim_1 - processor_tb_singlecycle

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION

- Run Simulation

RTL ANALYSIS

- Run Linter
- Open Elaborated Design

SYNTHESIS

- Run Synthesis
- Open Synthesized Design

IMPLEMENTATION

- Run Implementation
- Open Implemented Design

Scope Sources

- Design Sources (1)
 - processor_singlecycle (processor_singlecycle)
- Constraints
 - constrs_1
- Simulation Sources (2)
 - sim_1 (2)
 - processor_tb_singlecycle (tb_processor_singlecycle)
 - dut: processor_singlecycle (processor_singlecycle)
 - Unknown (1)
 - program_singlecycle.hex
 - Utility Sources
 - utils_1

Objects

Name	Value
clock	1
reset	0
memEn	0
memData[31:0]	0
memAddr[31:0]	0
gp[31:0]	00000090
a7[31:0]	00000000
a0[31:0]	00000090
i[31:0]	32
cycle_count[31:0]	77
WIDTH[31:0]	32
CLK_PERIOD[31:0]	20
TIMEOUT_CYCLES[31:0]	10000

tb_processor_singlecycle.vv

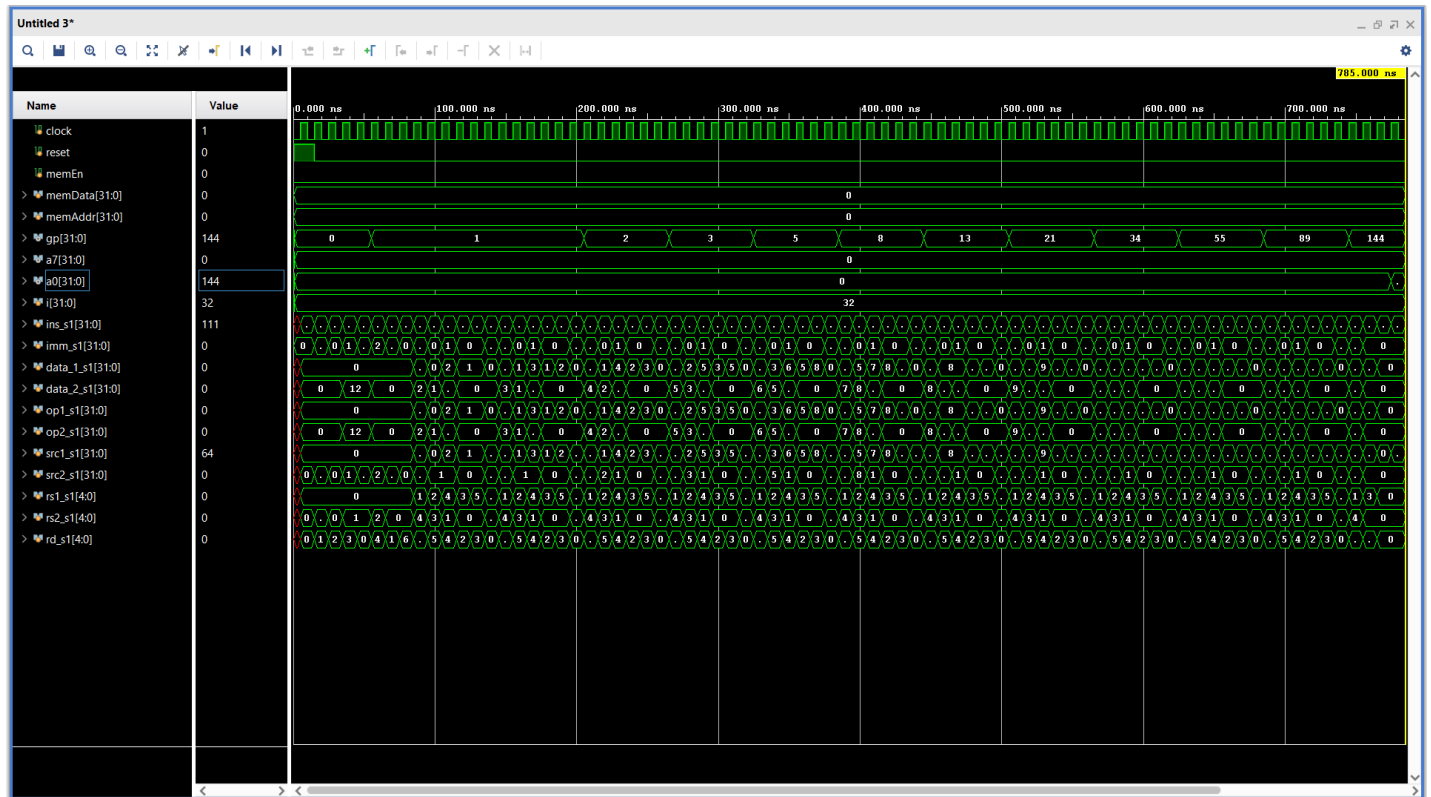
Name	Value
/processor_tb_singlecycle/memData[31:0]	00000000
memAddr[31:0]	00000000
gp[31:0]	144
a7[31:0]	00000000
a0[31:0]	144
i[31:0]	32
aluOp[3:0]	0
rs1[4:0]	0
rs2[4:0]	0
rd[4:0]	0

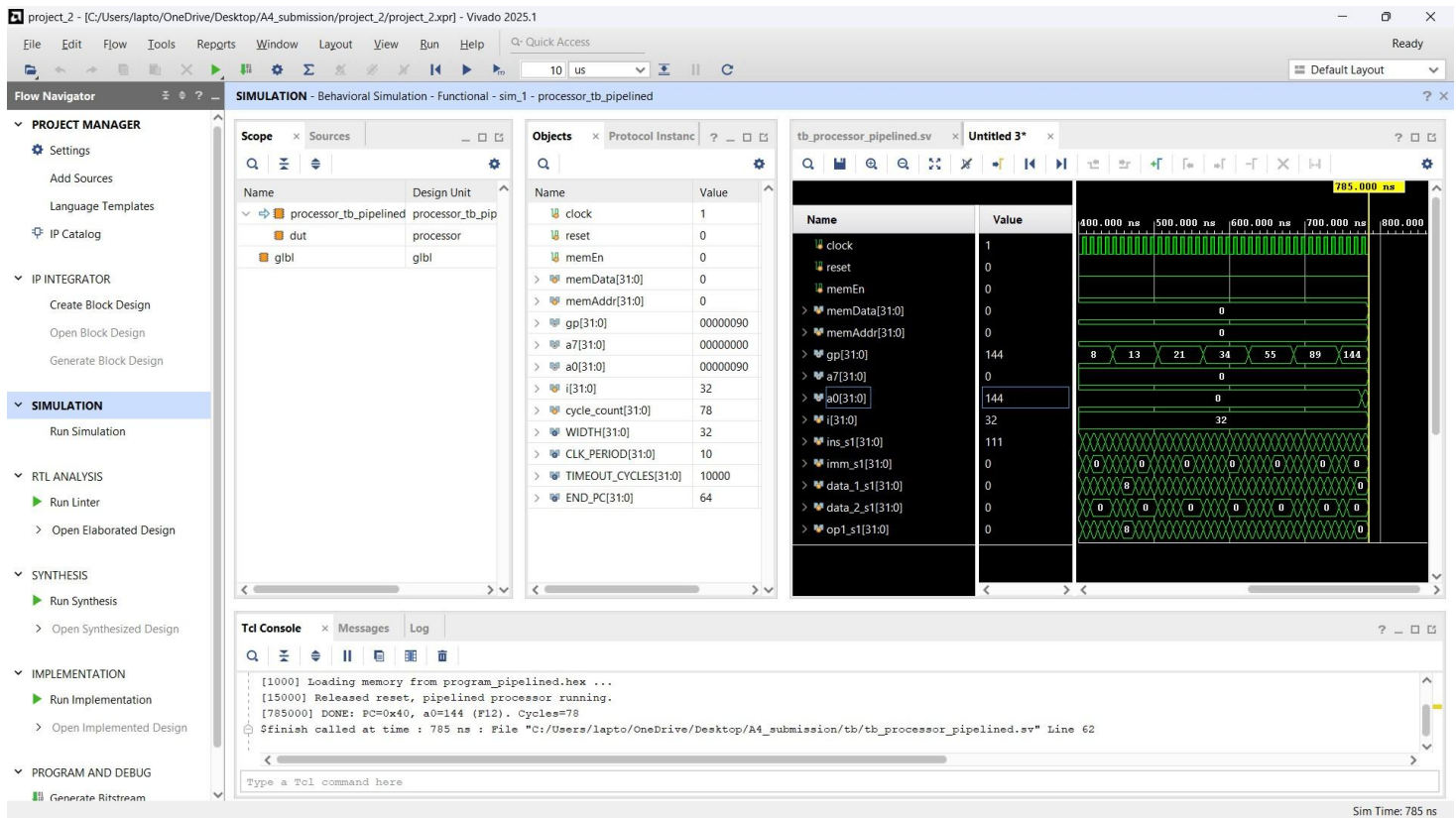
Tcl Console

```

Pass Through NonSizing Optimizer
Completed static elaboration
INFO: [XDM 43-4323] No Change in HDL. Linking previously generated obj files to create kernel
INFO: [USF-XSim-69] 'elaborate' step finished in '1' seconds
Time resolution is 1 ps
[1000] Loading memory from program_singlecycle.hex ...
[30000] Released reset, single-cycle processor running.
run all
[1550000] DONE: PC=0x3c, a0=144 (P12). Cycles=77
$finish called at time : 1550 ns : File "C:/Users/lapto/OneDrive/Desktop/A4_submission/tb/tb_processor_singlecycle.vv" Line 59
  
```

➤ *Pipelined CPU (NO forwarding)*





6. Execution time comparison

The A4 instructions require using $CLK_PERIOD = 20\text{ ns}$ for the single-cycle simulation and $CLK_PERIOD = 10\text{ ns}$ for the pipelined simulation. The provided testbenches print the cycle count when the program reaches the final infinite loop with the correct $a0$ value.

Implementation	Program	Clock period	Cycles (to END_PC)	Total time
Single-cycle	No hazard-avoidance	20ns	77	1540ns
2-stage pipeline (no fwd)	Hazard-safe	10ns	78	780ns
2-stage pipeline (forwarding)	No hazard-avoidance	10ns	77	770ns

From these counts the pipelined design reduces execution time from 1540 ns to 780 ns ($\sim 1.97x$ speedup). With forwarding enabled, time is 770 ns ($\sim 2.00x$).

7. Maximum clock frequency comparison (Vivado Synthesis)

For synthesis, set $MEM_DEPTH = 256$ in the processor parameters and apply a 50 ns clock constraint. After running synthesis, use the WNS value from the Timing Summary to compute the maximum Frequency $f_{max} = 1 / (T_{constraint} - WNS)$ where $T_{constraint} = 50\text{ ns}$

Design	WNS (ns) from Vivado	max clock period	Max frequency f_{max}

Single-cycle	– 47.867 ns	97.867	10.218 MHz
2-stage pipeline	– 43.309 ns	93.309	10.717 MHz

Single-cycle Timing report

project_1 - [C:/Users/lapto/OneDrive/Desktop/A4_submission/project_1/project_1.xpr] - Vivado 2025.1

File Edit Flow Tools Reports Window Layout View Help Quick Access

Synthesis Complete

Default Layout

Flow Navigator

Open Block Design

Generate Block Design

SIMULATION

Run Simulation

RTL ANALYSIS

Run Linter

Open Elaborated Design

SYNTHESIS

Run Synthesis

Open Synthesized Design

Constraints Wizard

Edit Timing Constraints

Set Up Debug

Open Dataflow Design

Report Timing Summary

Report Clock Networks

Report Clock Interaction

Report Methodology

Report DRC

Report Noise

Report Utilization

Report Power

Schematic

Netlist

processor_singlecycle

Nets (210465)

Leaf Cells (207622)

Project Summary

Device

tb_processor_singlecycle.sv

Timing Constraints

Create Clock

Clocks (1)

Create Clock (1)

Create Generated Clock

Rename Auto-Derived (

Set Clock Latency (0)

Set Clock Uncertainty (0)

Set Clock Groups (0)

Position

Clock Name

Period (ns)

Rise At (ns)

Fall At (ns)

Add Clock

Source Objects

Source File

Scope

1

clock

50.000

0.000

25.000

☒

[get_ports-filter

<unsaved co

Double click to create a Create Clock constraint

All Constraints

Position

Comm...

Scoped ..

<unsaved constraints>

1

create_clk

Apply

Cancel

Path Properties

Path 1

Summary

General

Properties

Report

Cells

Nets

Net Segmen

Tcl Console

Messages

Log

Reports

Design Runs

Timing

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Methodology Summary

Check Timing (120)

Intra-Clock Paths

clock

Setup -47.867 ns (10)

Timing Summary - clock

Setup

Worst Negative Slack (WNS): -47.867 ns

Total Negative Slack (TNS): -804480.049 ns

Number of Failing Endpoints: 17421

Total Number of Endpoints: 18413

Hold

Worst Hold Slack (WHS): 0.469 ns

Total Hold Slack (THS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 18413

Pulse Width

Worst Pulse Width Slack (WPWS): 24.500 ns

Total Pulse Width Negative Slack (TPWS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 17422

Timing constraints are not met.

2-stage pipeline Timing report

project_2 - [C:/Users/lapto/OneDrive/Desktop/A4_submission/project_2/project_2.xpr] - Vivado 2025.1

File Edit Flow Tools Reports Window Layout View Help Quick Access

Synthesis Complete

Default Layout

Flow Navigator

Create Block Design

Open Block Design

Generate Block Design

SIMULATION

Run Simulation

RTL ANALYSIS

Run Linter

Open Elaborated Design

SYNTHESIS

Run Synthesis

Open Synthesized Design

Constraints Wizard

Edit Timing Constraints

Set Up Debug

Open Dataflow Design

Report Timing Summary

Report Clock Networks

Report Clock Interaction

Report Methodology

Report DRC

Report Noise

Report Utilization

Report Power

Schematic

Netlist

processor

Nets (210900)

Leaf Cells (208057)

Project Summary

Device

tb_processor_pipeline.sv

Timing Constraints

Create Clock

Clocks (1)

Create Clock (1)

Create Generated Clock

Rename Auto-Derived (

Set Clock Latency (0)

Set Clock Uncertainty (0)

Set Clock Groups (0)

Position

Clock Name

Period (ns)

Rise At (ns)

Fall At (ns)

Add Clock

Source Objects

Source File

Scope

1

clock

50.000

0.000

25.000

☒

[get_ports-filter

<unsaved co

Double click to create a Create Clock constraint

All Constraints

Position

Command

Scce

<unsaved constraints>

Apply

Cancel

Simulation Object Properties

Select an object to see properties.

Tcl Console

Messages

Log

Reports

Design Runs

Timing

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Methodology Summary

Check Timing (120)

Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

Timing Summary - clock

Setup

Worst Negative Slack (WNS): -43.309 ns

Total Negative Slack (TNS): -11295.810 ns

Number of Failing Endpoints: 268

Total Number of Endpoints: 18680

Hold

Worst Hold Slack (WHS): 0.201 ns

Total Hold Slack (THS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 18680

Pulse Width

Worst Pulse Width Slack (WPWS): 24.500 ns

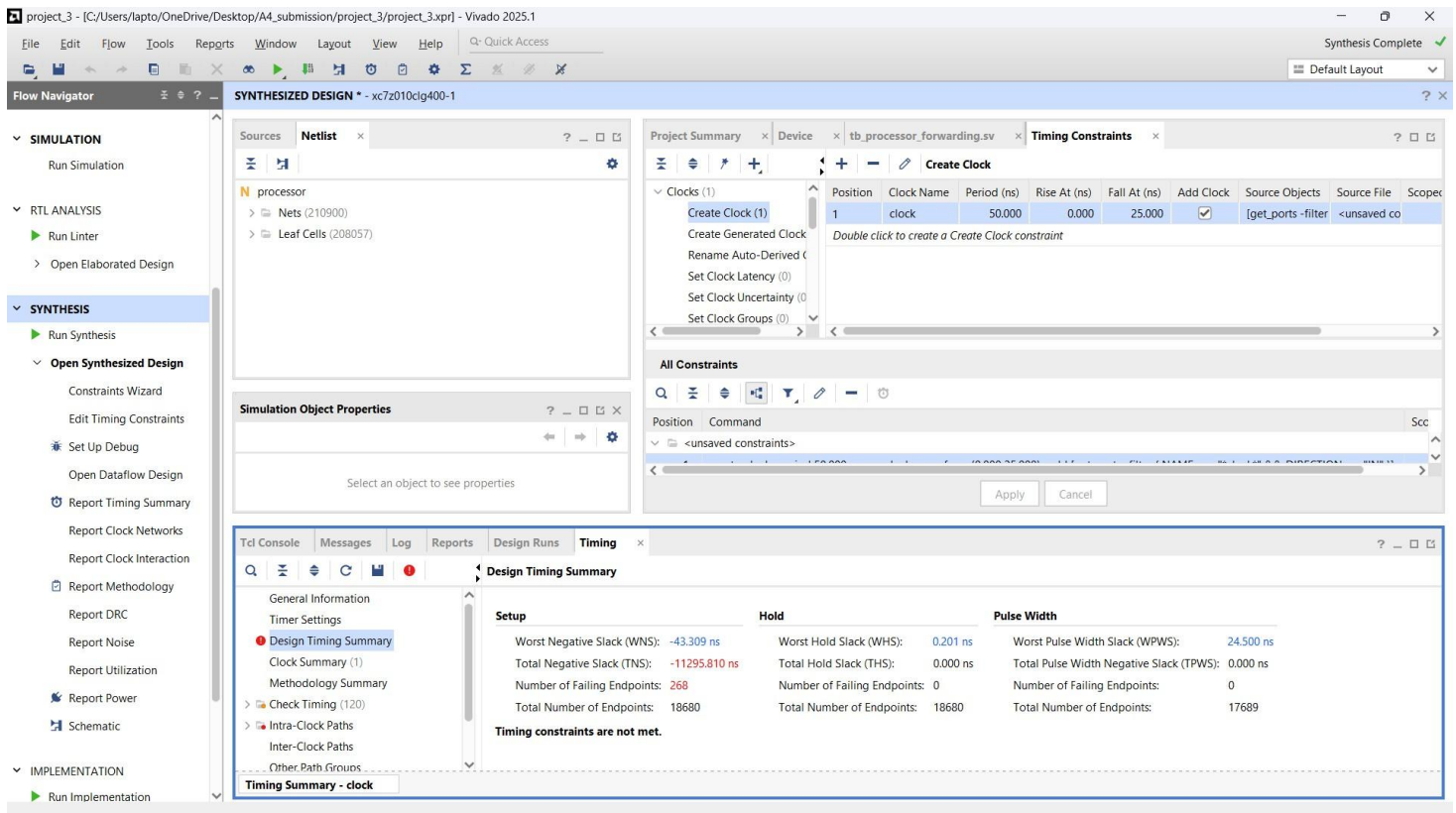
Total Pulse Width Negative Slack (TPWS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 17689

Timing constraints are not met.

2-stage pipeline (forwarding) Timing report



8. Optional forwarding implementation

When FORWARDING=1, Stage-1 operands (rs1/rs2) are compared against Stage-2 rd. If Stage-2 will write a register, the Stage-2 WB value (either ALU result/PC+4 or load data) is bypassed to Stage-1 to resolve Read-After-Write hazards without software NOPs.

Forwarding condition (conceptual):

if (regWriteEn_s2 && rd_s2!=0 && rd_s2==rsX_s1) opX_s1 = wb_final_s2;

9. Performance comment

The 2-stage pipeline improves performance primarily by allowing a shorter clock period and overlapping instruction execution. Without forwarding, performance depends on software scheduling / NOP insertion. With forwarding enabled, the same program can run correctly without hazard-avoidance instructions and with slightly fewer cycles.